

```
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '25mb',
    },
  },
};

const MODELS = {
  '2.1': 'openai/gpt-oss-20b',
  '2.5': 'qwen/qwen3.6-27b',
  '3.5': 'openai/gpt-oss-120b'
};

export default async function handler(req,
res) {
  res.setHeader('Access-Control-Allow-
Origin', '*');
  res.setHeader('Access-Control-Allow-
Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-
Headers', 'Content-Type');

  if (req.method === 'OPTIONS') return
res.status(200).end();
  if (req.method !== 'POST') return
res.status(405).json({ error: 'Method not
allowed' });

  const apiKey = process.env.GROQ_API_KEY;
```

```

    if (!apiKey) return
    res.status(500).json({ error: 'API key
    tidak ditemukan' });

    // — WHISPER TRANSCRIPTION —
    if (req.body.whisper) {
      try {
        const { audioBase64, mimeType } =
        req.body;
        const audioBuffer =
        Buffer.from(audioBase64, 'base64');

        const formData = new FormData();
        const blob = new Blob([audioBuffer],
        { type: mimeType || 'audio/webm' });
        formData.append('file', blob,
        'audio.webm');
        formData.append('model', 'whisper-
        large-v3-turbo');
        formData.append('language', 'id');
        formData.append('response_format',
        'json');

        const response = await
        fetch('https://api.groq.com/openai/v1/audio
        /transcriptions', {
          method: 'POST',
          headers: { 'Authorization': `Bearer
        ${apiKey}` },
          body: formData
        });

        const data = await response.json();
        if (!response.ok) return

```

```

res.status(response.status).json({ error:
data?.error?.message || 'Whisper error' });
    return res.status(200).json({ text:
data.text });
    } catch(e) {
        return res.status(500).json({ error:
e.message });
    }
}

// — CHAT —
try {
    const { messages, system, mode } =
req.body;

    const hasImage = messages.some(m =>
Array.isArray(m.content));

    // Vision uses gpt-oss-120b, otherwise
use mode model
    const model = hasImage ? 'openai/gpt-
oss-120b' : (MODELS[mode] ||
MODELS['2.1']);

    const groqMessages = [
        { role: 'system', content: system ||
'Kamu adalah NatMind, asisten AI yang
cerdas dan membantu.' },
        ...messages.map(m => {
            if (Array.isArray(m.content)) {
                const parts = m.content.map(c =>
{
                    if (c.type === 'text') return {
type: 'text', text: c.text };

```

```

        if (c.type === 'image') return
        {
            type: 'image_url',
            image_url: { url:
`data:${c.source.media_type};base64,${c.sou
rce.data}` }
        };
        return null;
    }).filter(Boolean);
    return { role: m.role, content:
parts };
    }
    return { role: m.role, content:
m.content };
    })
    ];

    const response = await
fetch('https://api.groq.com/openai/v1/chat/
completions', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${apiKey}`
    },
    body: JSON.stringify({ model,
messages: groqMessages, max_tokens: 2048 })
    });

    const data = await response.json();
    if (!response.ok) return
res.status(response.status).json({ error:
data?.error?.message || 'Groq error' });

```

```
    let text = data?.choices?.
[0]?.message?.content || 'Tidak ada
respons';

    // Remove <think>...</think> tags from
deepseek reasoning model
    text = text.replace(/<think>[\s\S]*?
<\think>/g, '').trim();

    return res.status(200).json({ content:
[ { text } ] });
  } catch(e) {
    return res.status(500).json({ error:
e.message });
  }
}
```